Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «АПО ЭВМиС»

Тема: «Архитектура и программирование сопроцессора. Использование целочисленных и вещественных команд»

Выполнил:
Студент 4-го курса
Группы АС-66
Горобец А.В.
Проверил:
Кулик А.Д.

Брест 2026

Раздел 1. Использование целочисленных команд

Цель работы: изучение архитектуры и программной модели сопроцессора; практическая работа с системой целочисленных команд

1. Изучить следующие теоретические сведения: архитектура и программная модель сопроцессора – регистры и их назначение; форматы чисел, типовые команды, целочисленные команды (используя материалы лекций и методические
указания).
2. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и целочисленные команды
согласно варианту, выданному преподавателем (таблица 2.2). Оформить ее в виде
отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям.

Вариант 5

$$y = \begin{cases} 3x + 2y^2 - 5, & xy > 8; \\ \dfrac{xy + 7}{2x^2 + 4}, & xy < -12; \\ 4x^2 - 3y^2 + 4, & -12 \le xy \le 8. \end{cases}$$

```
default rel

%macro PRINT 2
    mov     eax, 1              ; sys_write
    mov     edi, 1              ; stdout
    lea     rsi, [%1]
    mov     edx, %2
    syscall
%endmacro

section .data
    msg_x    db "Введите целое x: "
    len_x    equ $ - msg_x
    msg_y    db "Введите целое y: "
    len_y    equ $ - msg_y
    msg_b1   db "Ветка 1: xy > 8", 10
    len_b1   equ $ - msg_b1
    msg_b2   db "Ветка 2: xy < -12", 10
    len_b2   equ $ - msg_b2
    msg_b3   db "Ветка 3: -12 <= xy <= 8", 10
    len_b3   equ $ - msg_b3
    msg_res  db "Результат f = "
    len_res  equ $ - msg_res

    c2       dd 2
    c3       dd 3
    c4       dd 4
    c5       dd 5
    c7       dd 7
```

```
    c8       dd 8
    cm12     dd -12

section .bss
    inbuf    resb 256
    outbuf   resb 32
    inptr    resq 1
    x        resd 1
    y        resd 1
    xy       resq 1
    den      resd 1
    res      resq 1

section .text
global _start

_start:
    lea     rax, [inbuf]
    mov     [inptr], rax

    PRINT   msg_x, len_x
    call    read_int
    mov     [x], eax

    PRINT   msg_y, len_y
    call    read_int
    mov     [y], eax

    call    calc

    PRINT   msg_res, len_res
    mov     rax, [res]
    call    print_int

    mov     eax, 60             ; sys_exit
    xor     edi, edi
    syscall

calc:
    fild    dword [x]
    fimul   dword [y]
    fistp   qword [xy]          ; xy = x*y

    fild    qword [xy]
    ficom   dword [c8]          ; сравнение xy с 8
    fstsw   ax
    fstp    st0
    sahf
    jbe     .not_b1             ; xy <= 8
    call    branch1
    ret
.not_b1:
    fild    qword [xy]
    ficom   dword [cm12]        ; сравнение xy с -12
    fstsw   ax
    fstp    st0
    sahf
    jae     .not_b2             ; xy >= -12
    call    branch2
    ret
.not_b2:
    call    branch3
    ret

branch1:
```

```
    PRINT   msg_b1, len_b1
    fild    dword [x]
    fimul   dword [c3]          ; 3x
    fild    dword [y]
    fimul   dword [y]           ; y^2
    fimul   dword [c2]          ; 2y^2
    faddp   st1, st0            ; 3x + 2y^2
    fisub   dword [c5]          ; - 5
    fistp   qword [res]
    ret

branch2:
    PRINT   msg_b2, len_b2
    fild    dword [x]
    fimul   dword [x]           ; x^2
    fimul   dword [c2]          ; 2x^2
    fiadd   dword [c4]          ; 2x^2 + 4
    fistp   dword [den]
    fild    qword [xy]
    fiadd   dword [c7]          ; xy + 7
    fidiv   dword [den]         ; / (2x^2 + 4)
    fisttp  qword [res]         ; с отбрасыванием дробной части
    ret
branch3:
    PRINT   msg_b3, len_b3
    fild    dword [y]
    fimul   dword [y]           ; y^2
    fimul   dword [c3]          ; 3y^2
    fchs                        ; -3y^2
    fild    dword [x]
    fimul   dword [x]           ; x^2
    fimul   dword [c4]          ; 4x^2
    faddp   st1, st0            ; 4x^2 - 3y^2
    fiadd   dword [c4]          ; + 4
    fistp   qword [res]
    ret


read_int:
    mov     rsi, [inptr]
    xor     r8d, r8d            ; признак минуса
    xor     eax, eax
.skip:
    movzx   ecx, byte [rsi]
    test    ecx, ecx
    jz      .refill
    cmp     ecx, '-'
    jne     .chk
    mov     r8d, 1
    inc     rsi
    jmp     .dig
.chk:
    mov     edx, ecx
    sub     edx, '0'
    cmp     edx, 9
    jbe     .dig
    inc     rsi
    jmp     .skip
.dig:
    movzx   ecx, byte [rsi]
    sub     ecx, '0'
    cmp     ecx, 9
    ja      .end
    imul    rax, rax, 10
    add     rax, rcx
    inc     rsi
    jmp     .dig
.refill:
    xor     eax, eax            ; sys_read
    xor     edi, edi            ; stdin
    lea     rsi, [inbuf]
```

```
    mov     edx, 255
    syscall
    test    rax, rax
    jle     .eof
    mov     byte [rsi + rax], 0
    xor     eax, eax
    jmp     .skip
.eof:
    mov     byte [rsi], 0
    xor     eax, eax
.end:
    mov     [inptr], rsi
    test    r8d, r8d
    jz      .ret
    neg     rax
.ret:
    ret


print_int:
    lea     rdi, [outbuf + 31]
    mov     byte [rdi], 10
    xor     r8d, r8d
    test    rax, rax
    jns     .conv
    neg     rax
    mov     r8d, 1
.conv:
    mov     r9d, 10
.loop:
    xor     edx, edx
    div     r9
    add     dl, '0'
    dec     rdi
    mov     [rdi], dl
    test    rax, rax
    jnz     .loop
    test    r8d, r8d
    jz      .out
    dec     rdi
    mov     byte [rdi], '-'
.out:
    lea     rdx, [outbuf + 32]
    sub     rdx, rdi
    mov     rsi, rdi
    mov     eax, 1
    mov     edi, 1
    syscall
    ret
```

STDIN
```
1 9
```

Output:
```
Введите целое x: Введите целое y: Ветка 1: xy > 8
Результат f = 160
```

STDIN
```
1 -17
```

Output:
```
Введите целое x: Введите целое y: Ветка 2: xy < -12
Результат f = -1
```

STDIN
```
1 7
```

Output:
```
Введите целое x: Введите целое y: Ветка 3: -12 <= xy <= 8
Результат f = -139
```

1. Изучить систему вещественных команд сопроцессора (используя материалы лекций и методические указания).

2. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и вещественные команды
согласно варианту, выданному преподавателем (таблица 2.2). Оформить ее в виде
отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям.

```
default rel

%macro PRINT 2
    mov     eax, 1              ; sys_write
    mov     edi, 1              ; stdout
    lea     rsi, [%1]
    mov     edx, %2
    syscall
%endmacro

section .data
    msg_x    db "Введите вещественное x: "
    len_x    equ $ - msg_x
    msg_y    db "Введите вещественное y: "
    len_y    equ $ - msg_y
    msg_b1   db "Ветка 1: xy > 8", 10
    len_b1   equ $ - msg_b1
    msg_b2   db "Ветка 2: xy < -12", 10
    len_b2   equ $ - msg_b2
    msg_b3   db "Ветка 3: -12 <= xy <= 8", 10
    len_b3   equ $ - msg_b3
    msg_res  db "Результат f = "
    len_res  equ $ - msg_res

    c2       dq 2.0
    c3       dq 3.0
    c4       dq 4.0
    c5       dq 5.0
    c7       dq 7.0
    c8       dq 8.0
    cm12     dq -12.0
    ten      dq 10.0
    tenk     dq 10000.0

section .bss
    inbuf    resb 256
    outbuf   resb 32
    inptr    resq 1
    x        resq 1
    y        resq 1
    xy       resq 1
    den      resq 1
    res      resq 1
    divd     resq 1
    tmpd     resd 1
    tmpq     resq 1

section .text
global _start

_start:
    lea     rax, [inbuf]
    mov     [inptr], rax
```

```
    PRINT   msg_x, len_x
    call    read_real
    fstp    qword [x]

    PRINT   msg_y, len_y
    call    read_real
    fstp    qword [y]

    call    calc

    PRINT   msg_res, len_res
    fld     qword [res]
    call    print_real

    mov     eax, 60             ; sys_exit
    xor     edi, edi
    syscall

calc:
    fld     qword [x]
    fmul    qword [y]
    fstp    qword [xy]          ; xy = x*y

    fld     qword [xy]
    fcomp   qword [c8]          ; сравнение xy с 8
    fstsw   ax
    sahf
    jbe     .not_b1             ; xy <= 8
    call    branch1
    ret
.not_b1:
    fld     qword [xy]
    fcomp   qword [cm12]        ; сравнение xy с -12
    fstsw   ax
    sahf
    jae     .not_b2             ; xy >= -12
    call    branch2
    ret
.not_b2:
    call    branch3
    ret

branch1:
    PRINT   msg_b1, len_b1
    fld     qword [c3]
    fmul    qword [x]           ; 3x
    fld     qword [y]
    fmul    st0, st0            ; y^2
    fmul    qword [c2]          ; 2y^2
    faddp   st1, st0            ; 3x + 2y^2
    fsub    qword [c5]          ; - 5
    fstp    qword [res]
    ret

branch2:
    PRINT   msg_b2, len_b2
    fld     qword [x]
    fmul    st0, st0            ; x^2
    fmul    qword [c2]          ; 2x^2
    fadd    qword [c4]          ; 2x^2 + 4
    fstp    qword [den]
    fld     qword [xy]
    fadd    qword [c7]          ; xy + 7
    fdiv    qword [den]         ; / (2x^2 + 4)
    fstp    qword [res]
```

```
    ret

branch3:
    PRINT   msg_b3, len_b3
    fld     qword [y]
    fmul    st0, st0            ; y^2
    fmul    qword [c3]          ; 3y^2
    fchs                        ; -3y^2
    fld     qword [x]
    fmul    st0, st0            ; x^2
    fmul    qword [c4]          ; 4x^2
    faddp   st1, st0            ; 4x^2 - 3y^2
    fadd    qword [c4]          ; + 4
    fstp    qword [res]
    ret


read_real:
    mov     rsi, [inptr]
    xor     r8d, r8d            ; признак минуса
    fld1
    fstp    qword [divd]        ; делитель дробной части = 1
    fldz                        ; накопитель
.skip:
    movzx   ecx, byte [rsi]
    test    ecx, ecx
    jz      .refill
    cmp     ecx, '-'
    jne     .chk
    mov     r8d, 1
    inc     rsi
    jmp     .int
.chk:
    mov     edx, ecx
    sub     edx, '0'
    cmp     edx, 9
    jbe     .int
    inc     rsi
    jmp     .skip
.int:                           ; целая часть
    movzx   ecx, byte [rsi]
    sub     ecx, '0'
    cmp     ecx, 9
    ja      .dot
    mov     [tmpd], ecx
    fmul    qword [ten]
    fild    dword [tmpd]
    faddp   st1, st0
    inc     rsi
    jmp     .int
.dot:
    movzx   ecx, byte [rsi]
    cmp     ecx, '.'
    je      .frac_start
    cmp     ecx, ','
    jne     .done
.frac_start:
    inc     rsi
.frac:                          ; дробная часть
    movzx   ecx, byte [rsi]
    sub     ecx, '0'
    cmp     ecx, 9
    ja      .done
    mov     [tmpd], ecx
    fmul    qword [ten]
    fild    dword [tmpd]
    faddp   st1, st0
    fld     qword [divd]
    fmul    qword [ten]
    fstp    qword [divd]
    inc     rsi
```

```
    jmp     .frac
.refill:
    xor     eax, eax            ; sys_read
    xor     edi, edi            ; stdin
    lea     rsi, [inbuf]
    mov     edx, 255
    syscall
    test    rax, rax
    jle     .eof
    mov     byte [rsi + rax], 0
    jmp     .skip
.eof:
    mov     byte [rsi], 0
.done:
    fdiv    qword [divd]
    test    r8d, r8d
    jz      .ret
    fchs
.ret:
    mov     [inptr], rsi
    ret


print_real:
    xor     r8d, r8d
    ftst
    fstsw   ax
    sahf
    jae     .pos
    fchs
    mov     r8d, 1
.pos:
    fmul    qword [tenk]
    fistp   qword [tmpq]        ; округление до 1/10000
    mov     rax, [tmpq]
    test    rax, rax
    jnz     .nz
    xor     r8d, r8d            ; нет знака у нуля
.nz:
    xor     edx, edx
    mov     r9d, 10000
    div     r9                  ; RAX - целая часть, RDX - дробная
    mov     r12, rax
    mov     r10, rdx
    lea     rdi, [outbuf + 31]
    mov     byte [rdi], 10
    mov     ecx, 4
.f:
    mov     rax, r10
    xor     edx, edx
    mov     r9d, 10
    div     r9
    mov     r10, rax
    add     dl, '0'
    dec     rdi
    mov     [rdi], dl
    dec     ecx
    jnz     .f
    dec     rdi
    mov     byte [rdi], '.'
    mov     rax, r12
.i:
    xor     edx, edx
    mov     r9d, 10
    div     r9
    add     dl, '0'
    dec     rdi
    mov     [rdi], dl
    test    rax, rax
    jnz     .i
    test    r8d, r8d
```

```
    jz      .out
    dec     rdi
    mov     byte [rdi], '-'
.out:
    lea     rdx, [outbuf + 32]
    sub     rdx, rdi
    mov     rsi, rdi
    mov     eax, 1
    mov     edi, 1
    syscall
    ret
```

STDIN
1 7

Output:
Введите вещественное x: Введите вещественное y: Ветка 3: -12 <= xy <= 8
Результат f = -139.0000

STDIN
1 9

Output:
Введите вещественное x: Введите вещественное y: Ветка 1: xy > 8
Результат f = 160.0000

STDIN
1 -17

Output:
Введите вещественное x: Введите вещественное y: Ветка 2: xy < -12
Результат f = -1.6667

Вывод: я изучил архитектуру и программную модель сопроцессора, а также систему целочисленных и вещественных команд; составил и отладил программы вычисления функции по варианту 5, протестировал их по всем условиям.